Week 2: DATA & RETRIEVAL

Connecting LLMs with external data sources through retrieval systems – a crucial skill for building practical GenAI applications.

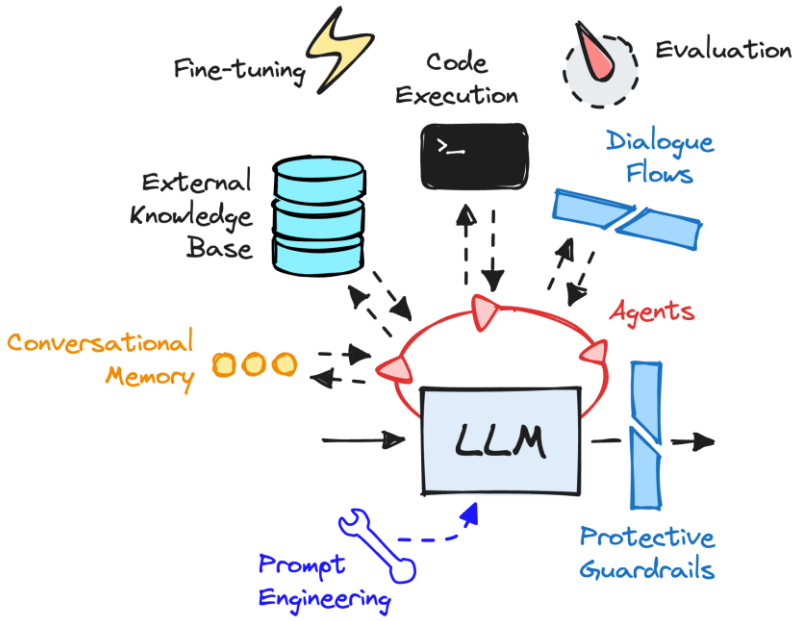
Topic 1: LLM Ecosystem & Limitations

What to Explore:

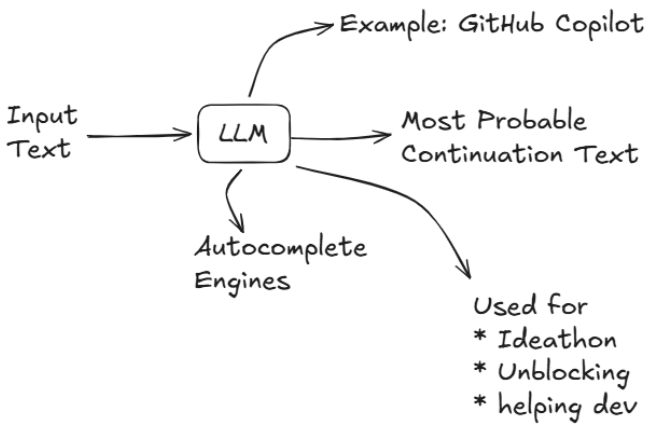
- Why standalone LLMs fall short in real applications
- Knowledge cutoffs, hallucinations, and domain-specific limitations
- The ecosystem of tools that complement LLMs

LLMs Are Not All You Need:

- LLM is a black box that does autocomplete very well.
- They are unpredictable, highly hallucinative, and unable to provide up-to-date information.



- LLMs completely depend on the ecosystem of tooling (tools being built around them).



- Feed input text to the LLMs and you get the most probable continuation text.
- LLMs are just like an autocomplete engine.

- Local LLMs:
    - To locally develop and deploy our own LLM, we need:
        - 7B parameter or larger model to get good responses.
        - ~28GM of memory (GPU memory; otherwise LLM will be slow)
    - We can reduce the model size by:
        - Using Hugging Face Transformers supports quantization of models.

<u>Hallucinations:</u>

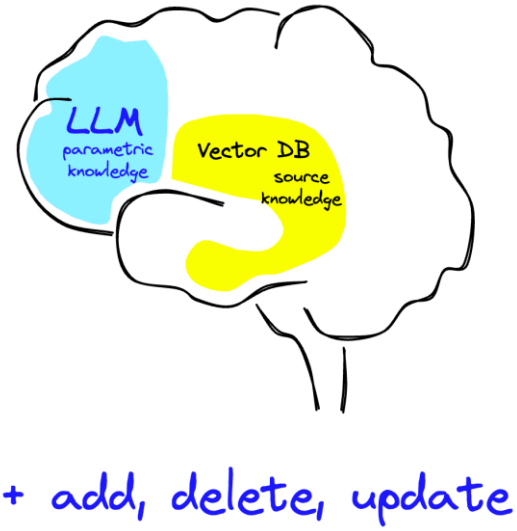
A well-known and significant problem of the LLMs. We can never eliminate hallucinations, but we can get close to preventing it.

<u>Prompt Engineering:</u>

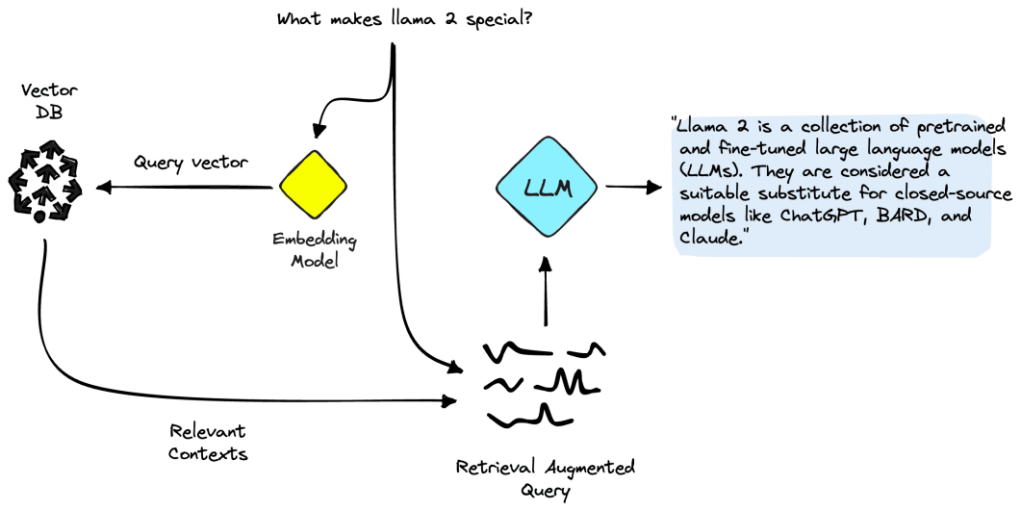
- A prompt consists of 2 or more components:
    - Instructions – tells the model what to do, and how to behave
    - External Information
    - One/Few-shot prompting
    - User input
    - Output indicator
- With prompt engineering, the performance of the LLM can be significantly improved

<u>Retrieval Augmented Generation:</u>

- LLM alone operates in parametric knowledge.
- The knowledge stored within the model parameters, which the model encodes during model training, is called the parametric knowledge.
- Parametric knowledge is learned from the training dataset.
- It is frozen in time as the model will not be trained past the cutoff date of the training data.
- Using source knowledge, the model can stay updated with all the information that is feed via our knowledge base.



- Context stuffing (keep on adding more context to the window) will quickly degrade the performance of the LLM.
- Therefore, we should minimize our context sizes and place the most important information and instructions at the beginning and end of our prompts.
- Rather than context stuff, it is best to insert relevant information into our prompts selectively. Commonly, we achieve using **R**etrieval **A**ugmented **G**eneration (RAG).



- The user's query is passed an embedding model and that is used to retrieve information from a vector database and pass that to the LLM by the input prompt.

<u>Conversational Memory</u>

- We need this conversational memory for any conversation used in chatbots.
- **Idea:** Rather than sending just the most recent interaction to the LLM, we send a history of interactions and the most recent on to the LLM (like a chat log).
- **The simplest form** of conversational memory is to store everything in raw form.

<u>Agents</u>

- Logical wrapper around LLM that allows to add several features to AI systems
- ReAct framework – LLM should go through "thinking loops" where they Reason and Act.
- By allowing LLMs to generate reasoning steps followed by tool-enhanced action steps, we unlock a seemingly infinite realm of possibility.
- AI developers build agent tooling with LangChain. Llama-index, Haystack, OpenAI Function Calling.

<u>Guardrail</u>

- A guardrail is a semi or fully deterministic shield that detects when an interaction triggers a particular condition set.
- If the condition is satisfied, then the conversational flow can be started an actions can be taken – which is similar to how an agent can decide to take an action, but this time, the guardrails decide what to do.
- The most robust framework for developing guardrails is NVIDIA's NeMo Guardrails.
- A canonical form is like a classification of user intent.